



Approximation Algorithms for Big Data

Lecture 5: Kernels in High Dimensions

Spring 2026

Instructor: Tal Wagner

Administrative Announcements

- Next week: one-hour class

Administrative Announcements

- Next week: one-hour class
- HW1 grades out tonight; HW2 to be released tonight, you have 3 weeks for it
 - From HW2: points will be deducted for failing to follow formatting guidelines

Administrative Announcements

- Next week: one-hour class
- HW1 grades out tonight; HW2 to be released tonight, you have 3 weeks for it
 - From HW2: points will be deducted for failing to follow formatting guidelines
- “Engagement” grade, for those not attending class regularly:
 - No written material or code accepted
 - Video recordings:
 - Ideally: will be useful to present/future students in the class
 - Expected: face and voice
 - Will be graded for quantity, depth, accuracy, overall quality
 - Other ideas? Be creative while keeping the intended goal in mind

Administrative Announcements

- Next week: one-hour class
- HW1 grades out tonight; HW2 to be released tonight, you have 3 weeks for it
 - From HW2: points will be deducted for failing to follow formatting guidelines
- “Engagement” grade, for those not attending class regularly:
 - No written material or code accepted
 - Video recordings:
 - Ideally: will be useful to present/future students in the class
 - Expected: face and voice
 - Will be graded for quantity, depth, accuracy, overall quality
 - Other ideas? Be creative while keeping the intended goal in mind
- Note on emails

Recap

- We covered: Probabilistic embedding into trees and quadtrees
 - Background and definitions
 - Quadtrees:
 - Distortion analysis
 - Approximate nearest neighbor search
 - L1-embedding
 - Earth Mover Distance (EMD)

Kernels

As similarity measures in high dimensions

Kernels

- A **kernel** is a map $k: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ that measures **similarity** between pairs of points

Kernels

- A **kernel** is a map $k: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ that measures **similarity** between pairs of points
- Some commonly used kernels:
 - **Gaussian**: $k(x, y) = e^{-\|x-y\|_2^2/2\sigma^2}$
 - **Laplacian**: $k(x, y) = e^{-\|x-y\|_1/\sigma}$ or $k(x, y) = e^{-\|x-y\|_2/\sigma}$
 - **Polynomial** (of degree p): $k(x, y) = \left(\frac{x^T y}{\sigma^2} + c\right)^p$

Kernels

- A **kernel** is a map $k: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ that measures **similarity** between pairs of points
- Some commonly used kernels:
 - **Gaussian**: $k(x, y) = e^{-\|x-y\|_2^2/2\sigma^2}$
 - **Laplacian**: $k(x, y) = e^{-\|x-y\|_1/\sigma}$ or $k(x, y) = e^{-\|x-y\|_2/\sigma}$
 - **Polynomial** (of degree p): $k(x, y) = \left(\frac{x^T y}{\sigma^2} + c\right)^p$
- σ is the **bandwidth** parameter
 - Controls smoothness
 - We will often assume w.l.o.g. that $\sigma = 1$ by scaling the points (but not always)

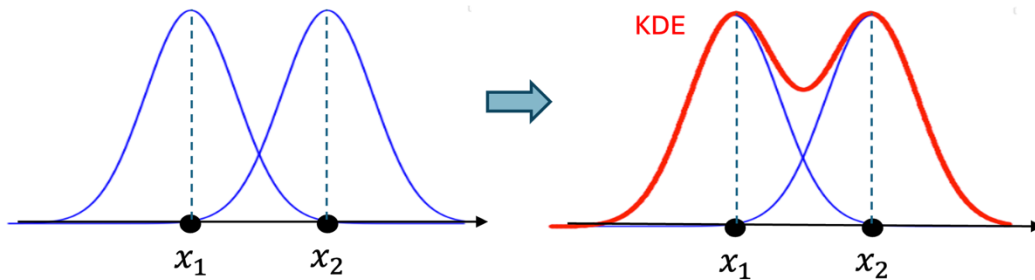
Why Kernels? (1) Kernel Density Estimation

Why Kernels? (1) Kernel Density Estimation

- Input: Set of n points $X \subset \mathbb{R}^d$
- Task: Given a query $y \in \mathbb{R}^d$, return $KDE_X(y) := \frac{1}{|X|} \sum_{x \in X} k(x, y)$

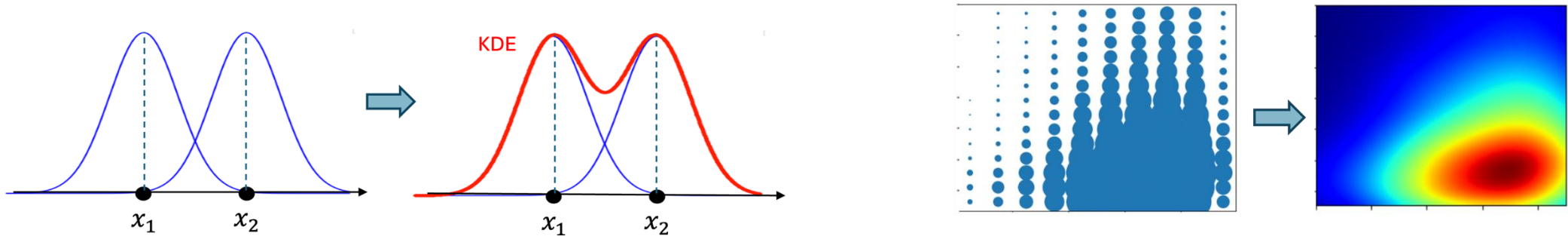
Why Kernels? (1) Kernel Density Estimation

- Input: Set of n points $X \subset \mathbb{R}^d$
- Task: Given a query $y \in \mathbb{R}^d$, return $KDE_X(y) := \frac{1}{|X|} \sum_{x \in X} k(x, y)$



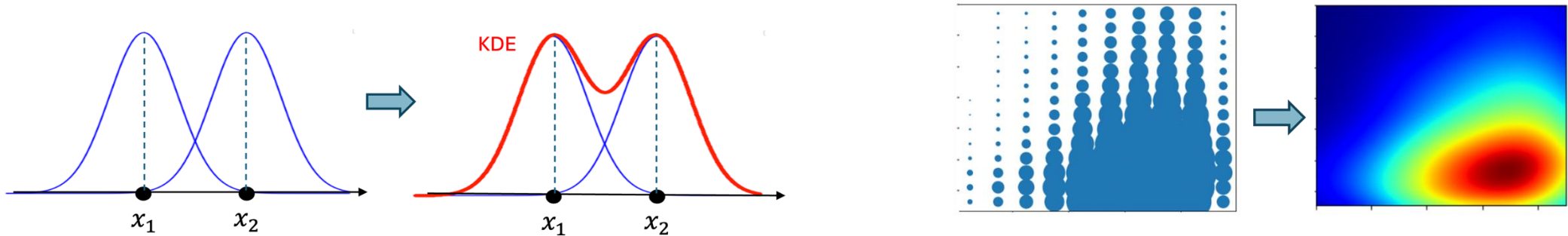
Why Kernels? (1) Kernel Density Estimation

- Input: Set of n points $X \subset \mathbb{R}^d$
- Task: Given a query $y \in \mathbb{R}^d$, return $KDE_X(y) := \frac{1}{|X|} \sum_{x \in X} k(x, y)$



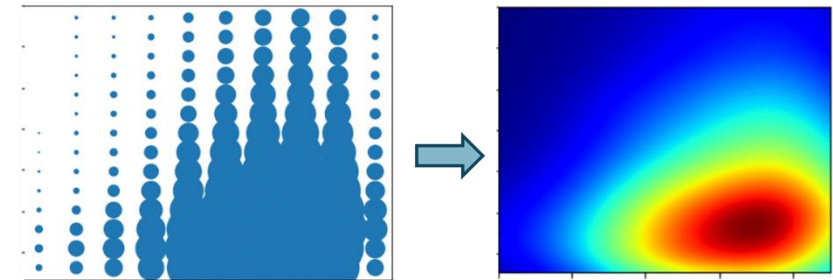
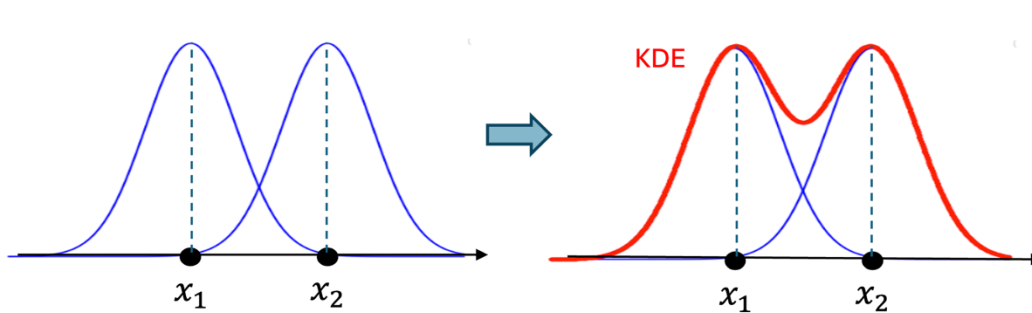
Why Kernels? (1) Kernel Density Estimation

- Input: Set of n points $X \subset \mathbb{R}^d$
- Task: Given a query $y \in \mathbb{R}^d$, return $KDE_X(y) := \frac{1}{|X|} \sum_{x \in X} k(x, y)$
- Goal: Answer queries approximately with high probability



Why Kernels? (1) Kernel Density Estimation

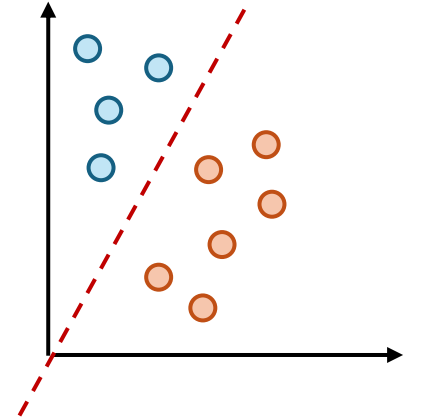
- Input: Set of n points $X \subset \mathbb{R}^d$
- Task: Given a query $y \in \mathbb{R}^d$, return $KDE_X(y) := \frac{1}{|X|} \sum_{x \in X} k(x, y)$
- Goal: Answer queries approximately with high probability
 - Exact computation time?



Why Kernels? (2) Machine Learning

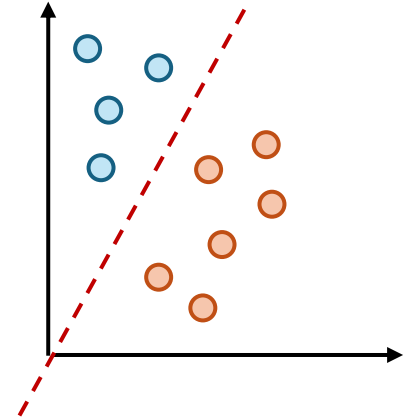
Why Kernels? (2) Machine Learning

- Support vector machines (SVMs) are ML models for learning linear separators between labeled classes



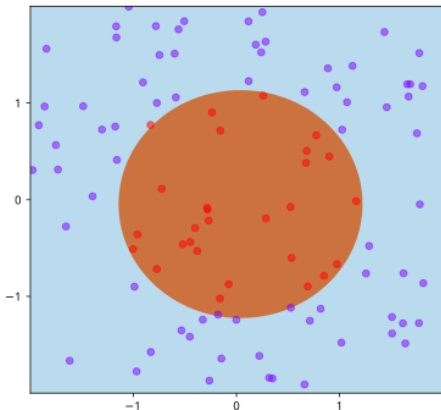
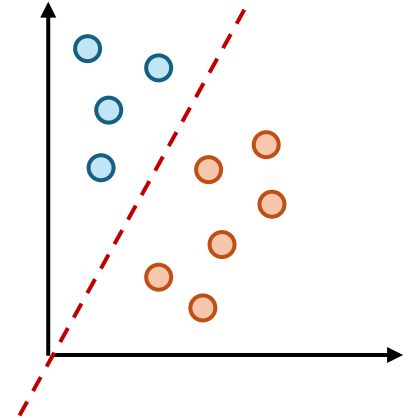
Why Kernels? (2) Machine Learning

- Support vector machines (SVMs) are ML models for learning linear separators between labeled classes
 - Input: Dataset $x_1, \dots, x_n \in \mathbb{R}^d$ with labels $\ell_1, \dots, \ell_n \in \{1, -1\}$
 - Goal: Compute an optimal linear separator



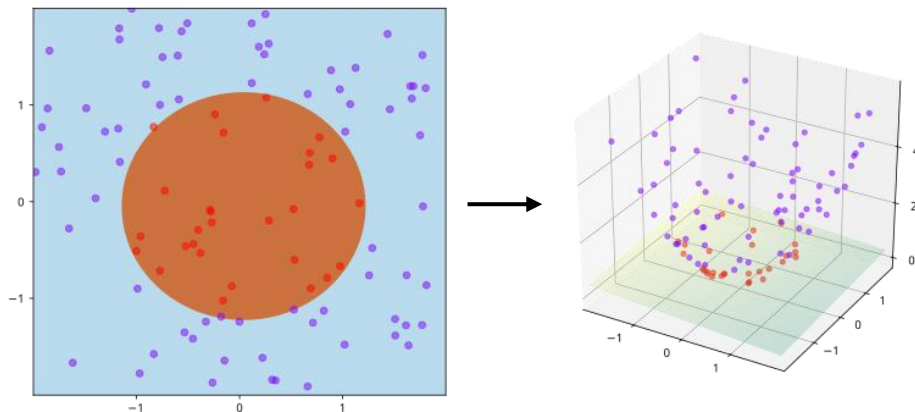
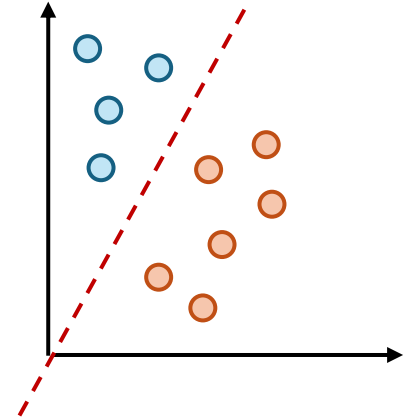
Why Kernels? (2) Machine Learning

- Support vector machines (SVMs) are ML models for learning linear separators between labeled classes
 - Input: Dataset $x_1, \dots, x_n \in \mathbb{R}^d$ with labels $\ell_1, \dots, \ell_n \in \{1, -1\}$
 - Goal: Compute an optimal linear separator
- What if the dataset is not linearly separable?



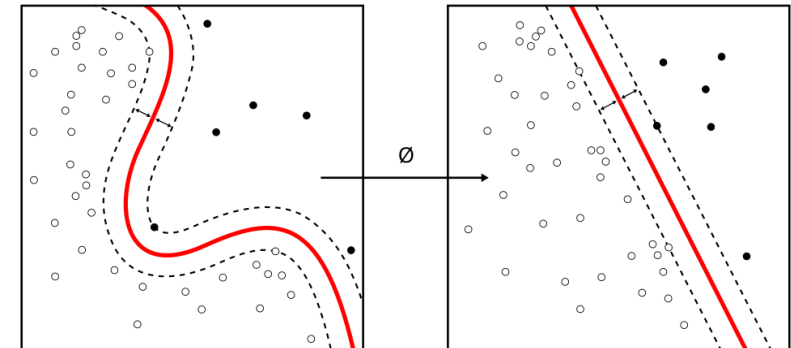
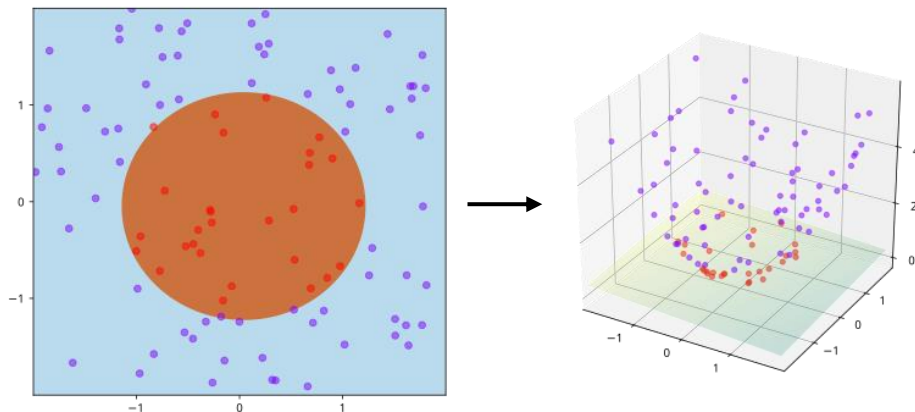
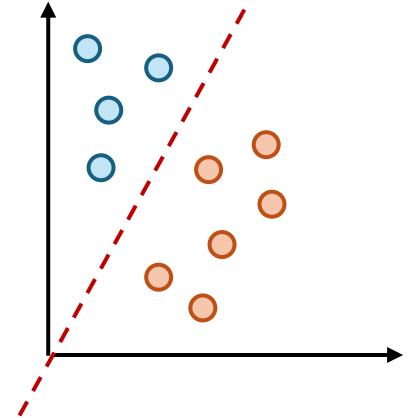
Why Kernels? (2) Machine Learning

- **Support vector machines (SVMs)** are ML models for learning linear separators between labeled classes
 - Input: Dataset $x_1, \dots, x_n \in \mathbb{R}^d$ with labels $\ell_1, \dots, \ell_n \in \{1, -1\}$
 - Goal: Compute an optimal linear separator
- What if the dataset is not linearly separable?
- Approach: Map the dataset to a different space where it is!



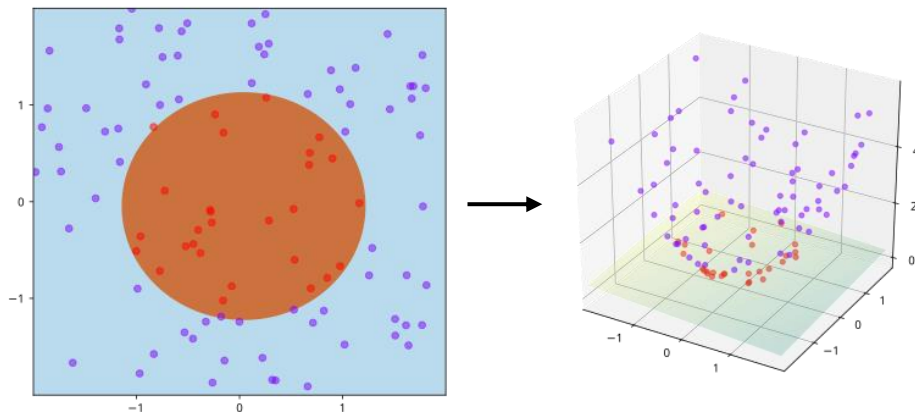
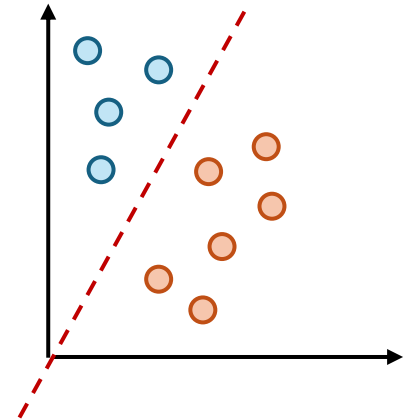
Why Kernels? (2) Machine Learning

- **Support vector machines (SVMs)** are ML models for learning linear separators between labeled classes
 - Input: Dataset $x_1, \dots, x_n \in \mathbb{R}^d$ with labels $\ell_1, \dots, \ell_n \in \{1, -1\}$
 - Goal: Compute an optimal linear separator
- **What if the dataset is not linearly separable?**
- Approach: Map the dataset to a different space where it is!

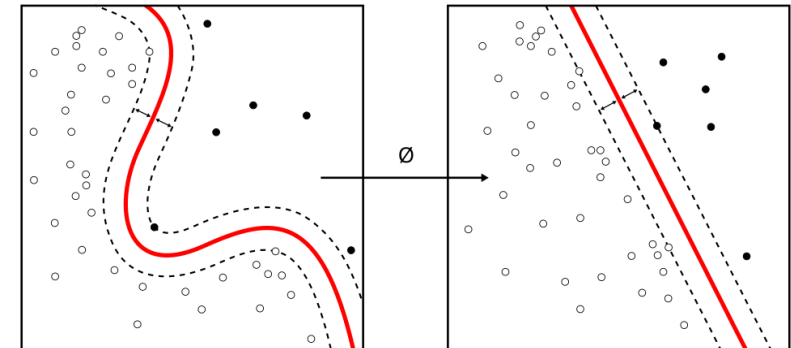


Why Kernels? (2) Machine Learning

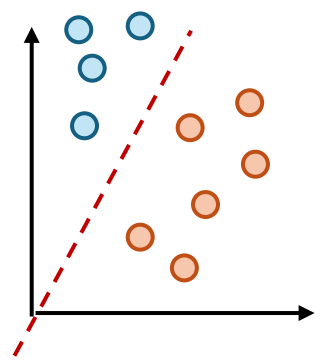
- **Support vector machines (SVMs)** are ML models for learning linear separators between labeled classes
 - Input: Dataset $x_1, \dots, x_n \in \mathbb{R}^d$ with labels $\ell_1, \dots, \ell_n \in \{1, -1\}$
 - Goal: Compute an optimal linear separator
- What if the dataset is not linearly separable?
- Approach: Map the dataset to a different space where it is!



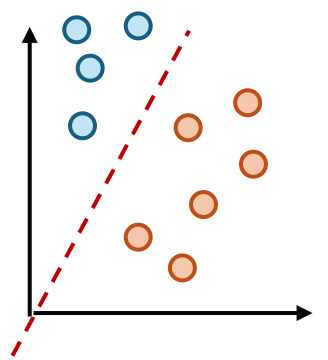
How?



”The Kernel Trick”

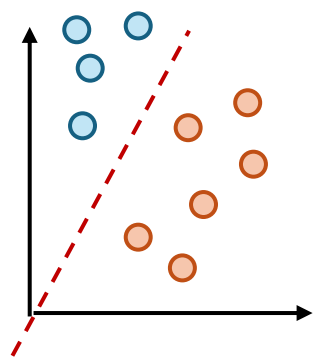


”The Kernel Trick”



- Finding an optimal linear separator means: $\min_{w \in \mathbb{R}^n} \|w\|^2 \quad s.t. \quad \forall_i, \ell_i \cdot w^T x_i \geq 1$

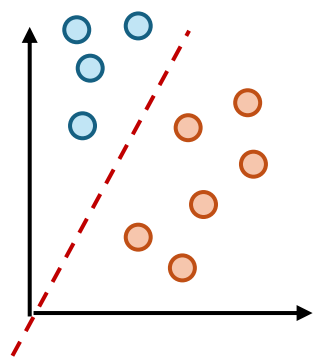
”The Kernel Trick”



- Finding an optimal linear separator means: $\min_{w \in \mathbb{R}^n} \|w\|^2 \quad s.t. \quad \forall_i, \ell_i \cdot w^T x_i \geq 1$

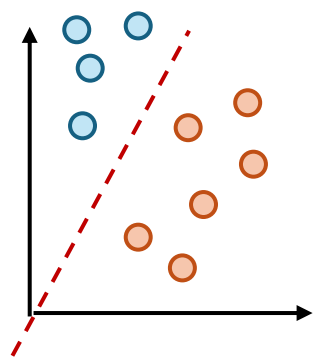
- Or equivalently, solving the dual problem: $\max_{\alpha \in \mathbb{R}^n: \alpha \geq 0} \left(\sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \ell_i \ell_j \cdot x_i^T x_j \right)$

”The Kernel Trick”



- Finding an optimal linear separator means: $\min_{w \in \mathbb{R}^n} \|w\|^2 \quad s.t. \quad \forall_i, \ell_i \cdot w^T x_i \geq 1$
- Or equivalently, solving the dual problem:
$$\max_{\alpha \in \mathbb{R}^n: \alpha \geq 0} \left(\sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \ell_i \ell_j \cdot x_i^T x_j \right)$$
- This accesses the dataset only by computing inner products

”The Kernel Trick”



- Finding an optimal linear separator means: $\min_{w \in \mathbb{R}^n} \|w\|^2 \quad s.t. \quad \forall_i, \ell_i \cdot w^T x_i \geq 1$

- Or equivalently, solving the dual problem: $\max_{\alpha \in \mathbb{R}^n: \alpha \geq 0} \left(\sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \ell_i \ell_j \cdot x_i^T x_j \right)$

- This accesses the dataset only by computing inner products
- We can replace the inner products with kernels and get **kernel SVM**:

$$\max_{\alpha \in \mathbb{R}^n: \alpha \geq 0} \left(\sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \ell_i \ell_j \cdot k(x_i, x_j) \right)$$

Kernels as Inner Products

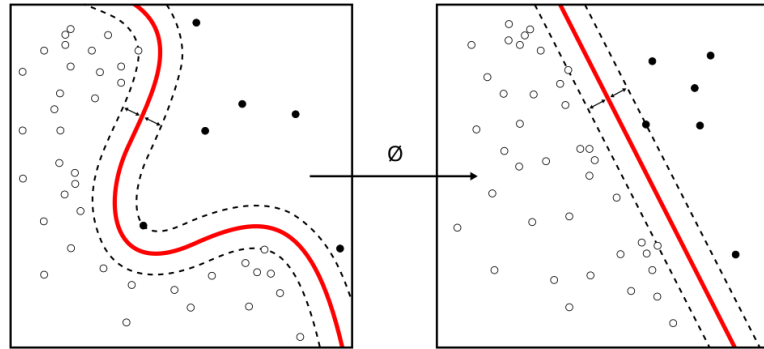
- Intuition: kernels are inner products in a transformed space

Kernels as Inner Products

- Intuition: kernels are inner products in a transformed space
- Meaning, there is a map $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $k(x, y) = \phi(x)^T \phi(y)$

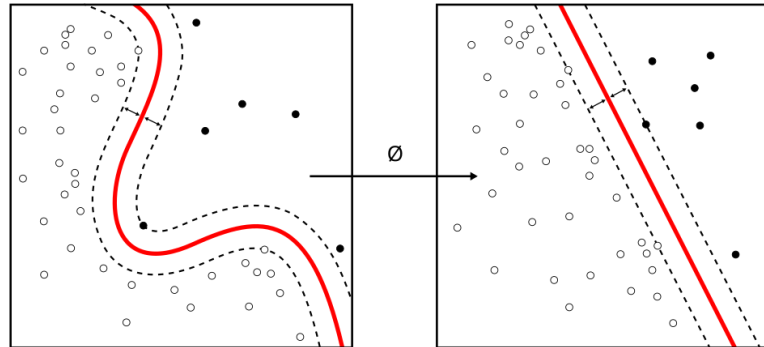
Kernels as Inner Products

- Intuition: kernels are inner products in a transformed space
- Meaning, there is a map $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $k(x, y) = \phi(x)^T \phi(y)$



Kernels as Inner Products

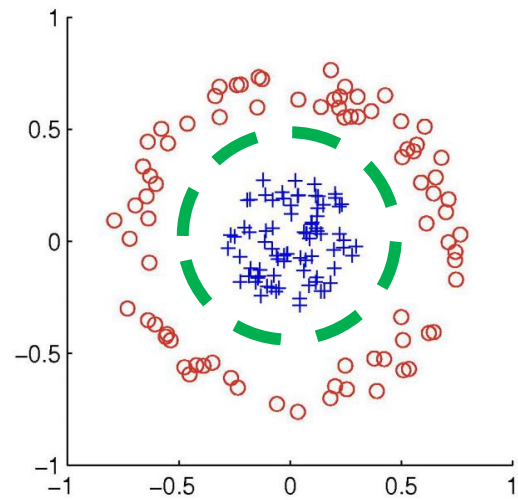
- Intuition: kernels are inner products in a transformed space
- Meaning, there is a map $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $k(x, y) = \phi(x)^T \phi(y)$



- However, D is typically **very large or infinite** and thus not useful algorithmically
 - Note that in kernel SVM we never needed to compute ϕ or realize D

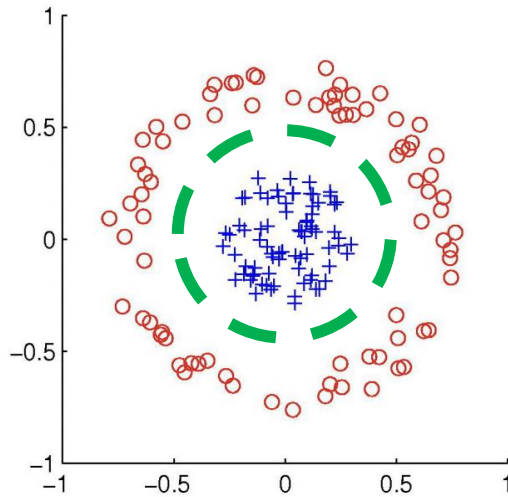
Kernels as Inner Products: Example

Original points in $\mathbb{R}^2$



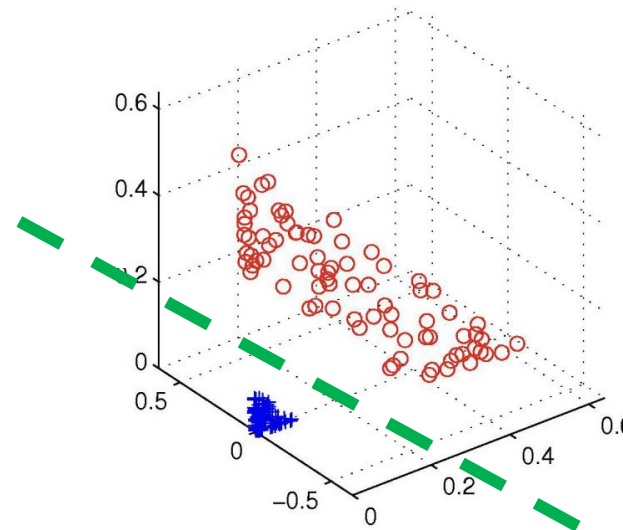
Kernels as Inner Products: Example

Original points in $\mathbb{R}^2$



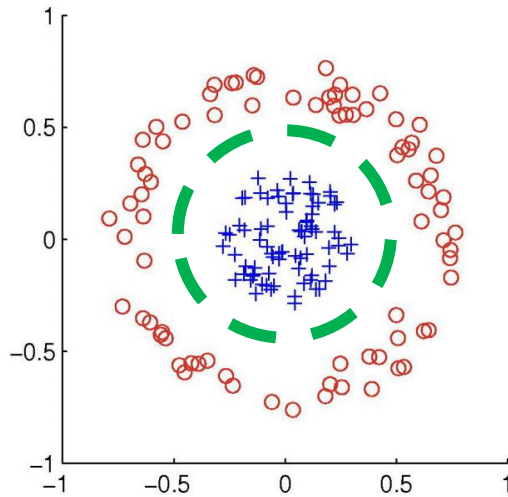
Mapping: $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^3$

$$\phi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$



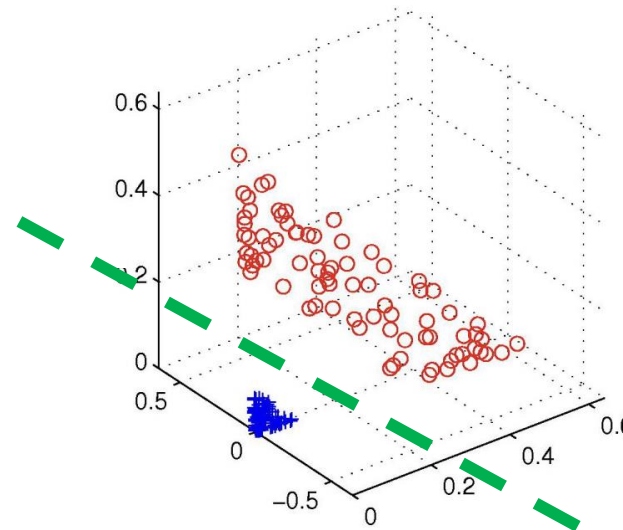
Kernels as Inner Products: Example

Original points in $\mathbb{R}^2$



Mapping: $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^3$

$$\phi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$



$$k(x, y) = \phi(x)^T \phi(y) = x_1^2 y_1^2 + 2x_1 x_2 y_1 y_2 + x_2^2 y_2^2 = (x_1 y_1 + x_2 y_2)^2$$

Kernel Features

Exact and approximate

Explicit Kernel Maps

- For simplicity, let's start with the homogeneous polynomial kernel: $k(x, y) = (x^T y)^p$

Explicit Kernel Maps

- For simplicity, let's start with the homogeneous polynomial kernel: $k(x, y) = (x^T y)^p$
- Can you come up with a $\phi_p: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $\phi_p(x)^T \phi_p(y) = (x^T y)^p$?

Explicit Kernel Maps

- For simplicity, let's start with the homogeneous polynomial kernel: $k(x, y) = (x^T y)^p$
- Can you come up with a $\phi_p: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $\phi_p(x)^T \phi_p(y) = (x^T y)^p$?

$$(x^T y)^p = \left(\sum_{i=1}^d x_i y_i \right)^p = \sum_{a_1, \dots, a_p \in [d]} \prod_{j=1}^p x_{a_j} y_{a_j}$$

Explicit Kernel Maps

- For simplicity, let's start with the homogeneous polynomial kernel: $k(x, y) = (x^T y)^p$
- Can you come up with a $\phi_p: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $\phi_p(x)^T \phi_p(y) = (x^T y)^p$?

$$(x^T y)^p = \left(\sum_{i=1}^d x_i y_i \right)^p = \sum_{a_1, \dots, a_p \in [d]} \prod_{j=1}^p x_{a_j} y_{a_j} = \phi_p(x)^T \phi_p(y)$$

$$\phi_p(x)_{a_1, \dots, a_p} = \prod_{j=1}^p x_{a_j}$$

Explicit Kernel Maps

- For simplicity, let's start with the homogeneous polynomial kernel: $k(x, y) = (x^T y)^p$
- Can you come up with a $\phi_p: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $\phi_p(x)^T \phi_p(y) = (x^T y)^p$?

$$(x^T y)^p = \left(\sum_{i=1}^d x_i y_i \right)^p = \sum_{a_1, \dots, a_p \in [d]} \prod_{j=1}^p x_{a_j} y_{a_j} = \phi_p(x)^T \phi_p(y)$$

$$\phi_p(x)_{a_1, \dots, a_p} = \prod_{j=1}^p x_{a_j}$$

- What is the target dimension? (Is it exponential in the source dimension?)

Explicit Kernel Maps: Gaussian Kernel

- Now, let's focus on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2}$

Explicit Kernel Maps: Gaussian Kernel

- Now, let's focus on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2}$
- Can you think of maps $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $\phi(x)^T \phi(y) = e^{-\|x-y\|_2^2}$?
 - Target dimension D can be infinite

Explicit Kernel Maps: Gaussian Kernel

- Now, let's focus on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2}$
- Can you think of maps $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $\phi(x)^T \phi(y) = e^{-\|x-y\|_2^2}$?
 - Target dimension D can be infinite
- One option (we will see another one soon): through Taylor expansion

Explicit Kernel Maps: Gaussian Kernel

- Now, let's focus on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2}$
- Can you think of maps $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $\phi(x)^T \phi(y) = e^{-\|x-y\|_2^2}$?
 - Target dimension D can be infinite
- One option (we will see another one soon): through Taylor expansion
- Recall: $e^z = \sum_{p=1}^{\infty} \frac{z^p}{p!}$

Explicit Kernel Maps: Gaussian Kernel

- Now, let's focus on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2}$
- Can you think of maps $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $\phi(x)^T \phi(y) = e^{-\|x-y\|_2^2}$?
 - Target dimension D can be infinite
- One option (we will see another one soon): through Taylor expansion

- Recall: $e^z = \sum_{p=1}^{\infty} \frac{z^p}{p!}$

- Hence: $e^{-\|x-y\|_2^2} = e^{-\|x\|_2^2} e^{-\|y\|_2^2} e^{x^T y} = e^{-\|x\|_2^2} e^{-\|y\|_2^2} \sum_{p=1}^{\infty} \frac{(x^T y)^p}{p!} =$???

Explicit Kernel Maps: Gaussian Kernel

- Now, let's focus on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2}$
- Can you think of maps $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $\phi(x)^T \phi(y) = e^{-\|x-y\|_2^2}$?
 - Target dimension D can be infinite
- One option (we will see another one soon): through Taylor expansion
- Recall:
$$e^z = \sum_{p=1}^{\infty} \frac{z^p}{p!}$$
- Hence:
$$e^{-\|x-y\|_2^2} = e^{-\|x\|_2^2} e^{-\|y\|_2^2} e^{x^T y} = e^{-\|x\|_2^2} e^{-\|y\|_2^2} \sum_{p=1}^{\infty} \frac{(x^T y)^p}{p!} = \phi(x)^T \phi(y)$$
- With:
$$\phi(x) = e^{-\|x\|_2^2} \cdot \left(\phi_0(x) \oplus \phi_1(x) \oplus \frac{1}{2} \phi_2(x) \oplus \dots \right) = e^{-\|x\|_2^2} \cdot \bigoplus_{p=1}^{\infty} \frac{1}{p!} \phi_p(x)$$

Approximate Kernel Maps

Approximate Kernel Maps

- Goal: Find a map $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that $f(x)^T f(y) \approx \phi(x)^T \phi(y) = e^{-\|x-y\|_2^2/2}$

Approximate Kernel Maps

- Goal: Find a map $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that $f(x)^T f(y) \approx \phi(x)^T \phi(y) = e^{-\|x-y\|_2^2/2}$
- This time, we want the target dimension m to be finite and computationally useful

Approximate Kernel Maps

- Goal: Find a map $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that $f(x)^T f(y) \approx \phi(x)^T \phi(y) = e^{-\|x-y\|_2^2/2}$
- This time, we want the target dimension m to be finite and computationally useful
- Does the explicit map from the previous slide lead to such f ?

$$e^{-\|x-y\|_2^2} = e^{-\|x\|_2^2} e^{-\|y\|_2^2} e^{x^T y} = e^{-\|x\|_2^2} e^{-\|y\|_2^2} \sum_{p=1}^{\infty} \frac{(x^T y)^p}{p!} = \phi(x)^T \phi(y)$$

Approximate Kernel Maps

- Goal: Find a map $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that $f(x)^T f(y) \approx \phi(x)^T \phi(y) = e^{-\|x-y\|_2^2/2}$
- This time, we want the target dimension m to be finite and computationally useful
- Does the explicit map from the previous slide lead to such f ?

$$e^{-\|x-y\|_2^2} = e^{-\|x\|_2^2} e^{-\|y\|_2^2} e^{x^T y} = e^{-\|x\|_2^2} e^{-\|y\|_2^2} \sum_{p=1}^{\infty} \frac{(x^T y)^p}{p!} = \phi(x)^T \phi(y)$$

- Next we'll see a different construction using randomization

Random Features for Kernels

- A random feature distribution $\mathcal{F}$ is a distribution over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all $x, y \in \mathbb{R}^d$:

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

Random Features for Kernels

- A random feature distribution $\mathcal{F}$ is a distribution over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all $x, y \in \mathbb{R}^d$:

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Does this remind you of anything?

Random Features for Kernels

- A random feature distribution $\mathcal{F}$ is a distribution over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all $x, y \in \mathbb{R}^d$:

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Does this remind you of anything?
- How do we use $\mathcal{F}$?

Random Features for Kernels

- A random feature distribution $\mathcal{F}$ is a distribution over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all $x, y \in \mathbb{R}^d$:

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Does this remind you of anything?
- How do we use $\mathcal{F}$? Draw $f_1, \dots, f_r \sim \mathcal{F}$, use $k(x, y) \approx \frac{1}{r} \sum_{j=1}^r f_j(x)^T f_j(y)$

Random Features for Kernels

- A random feature distribution $\mathcal{F}$ is a distribution over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all $x, y \in \mathbb{R}^d$:

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Does this remind you of anything?
- How do we use $\mathcal{F}$? Draw $f_1, \dots, f_r \sim \mathcal{F}$, use $k(x, y) \approx \frac{1}{r} \sum_{j=1}^r f_j(x)^T f_j(y)$
- Note: Every such $\mathcal{F}$ defines an explicit kernel map ϕ – how?

Random Features for Kernels

- A random feature distribution $\mathcal{F}$ is a distribution over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all $x, y \in \mathbb{R}^d$:

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Does this remind you of anything?
- How do we use $\mathcal{F}$? Draw $f_1, \dots, f_r \sim \mathcal{F}$, use $k(x, y) \approx \frac{1}{r} \sum_{j=1}^r f_j(x)^T f_j(y)$
- Note: Every such $\mathcal{F}$ defines an explicit kernel map ϕ – **how?**
- Let p denote the PDF of $\mathcal{F}$, and let $\phi(x) = \left(\sqrt{p(f)} \cdot f(x) \right)_{f \in \mathcal{F}} \in \mathbb{R}^{|\text{support}(\mathcal{F})|}$, then:

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x) \cdot f(y)] = \int_{\mathcal{F}} p(f) \cdot f(x) \cdot f(y) \cdot df = \phi(x)^T \phi(y)$$

Random Fourier Features

[Rahimi & Recht 2007]

Brief Background: Fourier Transform

Brief Background: Fourier Transform

- Let μ be a positive finite measure over $\mathbb{R}^d$
 - Note: This means μ can be normalized into a probability measure over $\mathbb{R}^d$

Brief Background: Fourier Transform

- Let μ be a positive finite measure over $\mathbb{R}^d$
 - Note: This means μ can be normalized into a probability measure over $\mathbb{R}^d$
- The Fourier transform of μ is:

$$F(x) = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T x} d\mu(\omega)$$

- Where here i denotes the imaginary unit

Brief Background: Fourier Transform

- Let μ be a positive finite measure over $\mathbb{R}^d$
 - Note: This means μ can be normalized into a probability measure over $\mathbb{R}^d$
- The Fourier transform of μ is:

$$F(x) = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T x} d\mu(\omega)$$

- Where here i denotes the imaginary unit
- Remark: if μ is a probability measure then $F(x) = \mathbb{E} \left[e^{-i \cdot \omega^T x} \right]$ - *is this familiar?*

Brief Background: Fourier Transform

- Let μ be a positive finite measure over $\mathbb{R}^d$
 - Note: This means μ can be normalized into a probability measure over $\mathbb{R}^d$
- The Fourier transform of μ is:

$$F(x) = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T x} d\mu(\omega)$$

- Where here i denotes the imaginary unit
- Remark: if μ is a probability measure then $F(x) = \mathbb{E} \left[e^{-i \cdot \omega^T x} \right]$ - *is this familiar?*
- This is nearly identical to the characteristic function of μ : $\Psi_\mu(x) = \mathbb{E} \left[e^{i \cdot \omega^T x} \right]$

Random Fourier Features [Rahimi & Recht '07]

Random Fourier Features [Rahimi & Recht '07]

- We focus (for now) on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2/2}$

Random Fourier Features [Rahimi & Recht '07]

- We focus (for now) on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2/2}$
- Fact: we can write it as the Fourier transform of the iid Gaussian measure over $\mathbb{R}^d$:

$$e^{-\|x-y\|_2^2/2} = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T(x-y)} \cdot \left(\frac{1}{\sqrt{2\pi}}\right)^d e^{-\|\omega\|_2^2/2} d\omega = \mathbb{E}_{\omega \sim N(0,1)^{\otimes d}} \left[e^{i \cdot \omega^T(x-y)} \right]$$

(“The Fourier transform of a Gaussian is a Gaussian”)

Random Fourier Features [Rahimi & Recht '07]

- We focus (for now) on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2/2}$
- Fact: we can write it as the Fourier transform of the iid Gaussian measure over $\mathbb{R}^d$:

$$e^{-\|x-y\|_2^2/2} = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T (x-y)} \cdot \left(\frac{1}{\sqrt{2\pi}}\right)^d e^{-\|\omega\|_2^2/2} d\omega = \mathbb{E}_{\omega \sim N(0,1)^{\otimes d}} \left[e^{i \cdot \omega^T (x-y)} \right]$$

(“The Fourier transform of a Gaussian is a Gaussian”)

$$= \mathbb{E}_{\omega \sim N(0,I)^{\otimes d}} \left[e^{i \cdot \omega^T x} \cdot \overline{e^{i \cdot \omega^T y}} \right]$$

Random Fourier Features [Rahimi & Recht '07]

- We focus (for now) on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2/2}$
- Fact: we can write it as the Fourier transform of the iid Gaussian measure over $\mathbb{R}^d$:

$$e^{-\|x-y\|_2^2/2} = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T (x-y)} \cdot \left(\frac{1}{\sqrt{2\pi}}\right)^d e^{-\|\omega\|_2^2/2} d\omega = \mathbb{E}_{\omega \sim N(0,1)^{\otimes d}} \left[e^{i \cdot \omega^T (x-y)} \right]$$

(“The Fourier transform of a Gaussian is a Gaussian”)

$$= \mathbb{E}_{\omega \sim N(0,I)^{\otimes d}} \left[e^{i \cdot \omega^T x} \cdot \overline{e^{i \cdot \omega^T y}} \right]$$

- and we almost have our random features, $f_\omega(x) = e^{i \cdot \omega^T x}$ where $\omega \sim N(0, I)^{\otimes d}$.

Random Fourier Features [Rahimi & Recht '07]

- We focus (for now) on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2/2}$
- Fact: we can write it as the Fourier transform of the iid Gaussian measure over $\mathbb{R}^d$:

$$e^{-\|x-y\|_2^2/2} = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T (x-y)} \cdot \left(\frac{1}{\sqrt{2\pi}}\right)^d e^{-\|\omega\|_2^2/2} d\omega = \mathbb{E}_{\omega \sim N(0,1)^{\otimes d}} \left[e^{i \cdot \omega^T (x-y)} \right]$$

(“The Fourier transform of a Gaussian is a Gaussian”)

$$= \mathbb{E}_{\omega \sim N(0,I)^{\otimes d}} \left[e^{i \cdot \omega^T x} \cdot \overline{e^{i \cdot \omega^T y}} \right]$$

- and we almost have our random features, $f_\omega(x) = e^{i \cdot \omega^T x}$ where $\omega \sim N(0, I)^{\otimes d}$.
- Why almost?

Random Fourier Features [Rahimi & Recht '07]

- We focus (for now) on the Gaussian kernel: $k(x, y) = e^{-\|x-y\|_2^2/2}$
- Fact: we can write it as the Fourier transform of the iid Gaussian measure over $\mathbb{R}^d$:

$$e^{-\|x-y\|_2^2/2} = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T (x-y)} \cdot \left(\frac{1}{\sqrt{2\pi}}\right)^d e^{-\|\omega\|_2^2/2} d\omega = \mathbb{E}_{\omega \sim N(0,1)^{\otimes d}} \left[e^{i \cdot \omega^T (x-y)} \right]$$

(“The Fourier transform of a Gaussian is a Gaussian”)

$$= \mathbb{E}_{\omega \sim N(0,I)^{\otimes d}} \left[e^{i \cdot \omega^T x} \cdot \overline{e^{i \cdot \omega^T y}} \right]$$

- and we almost have our random features, $f_\omega(x) = e^{i \cdot \omega^T x}$ where $\omega \sim N(0, I)^{\otimes d}$.
- Why almost?
- We'll see two ways to resolve this (both variants of RFF are widely used)

Real Valued RFF: 2-Dim Random Features

Real Valued RFF: 2-Dim Random Features

- We have: $k(x, y) = \mathbb{E}_{\omega} \left[e^{i \cdot \omega^T (x - y)} \right] = \mathbb{E}_{\omega} \left[\cos(\omega^T (x - y)) + i \cdot \sin(\omega^T (x - y)) \right]$

Real Valued RFF: 2-Dim Random Features

- We have: $k(x, y) = \mathbb{E}_{\omega} \left[e^{i \cdot \omega^T (x - y)} \right] = \mathbb{E}_{\omega} \left[\cos(\omega^T (x - y)) + i \cdot \sin(\omega^T (x - y)) \right]$
- We know that $k(x, y)$ is real-valued, hence the imaginary part on the right-side side is zero, hence: $e^{-\|x-y\|_2^2/2} = \mathbb{E}_{\omega} \left[\cos(\omega^T (x - y)) \right]$

Real Valued RFF: 2-Dim Random Features

- We have: $k(x, y) = \mathbb{E}_{\omega} \left[e^{i \cdot \omega^T (x - y)} \right] = \mathbb{E}_{\omega} \left[\cos(\omega^T (x - y)) + i \cdot \sin(\omega^T (x - y)) \right]$
- We know that $k(x, y)$ is real-valued, hence the imaginary part on the right-side side is zero, hence: $e^{-\|x-y\|_2^2/2} = \mathbb{E}_{\omega} \left[\cos(\omega^T (x - y)) \right]$
- Recall the trig sum-product formula: $\cos(\alpha - \beta) = \cos \alpha \cos \beta + \sin \alpha \sin \beta$

Real Valued RFF: 2-Dim Random Features

- We have: $k(x, y) = \mathbb{E}_{\omega} \left[e^{i \cdot \omega^T (x - y)} \right] = \mathbb{E}_{\omega} \left[\cos(\omega^T (x - y)) + i \cdot \sin(\omega^T (x - y)) \right]$
- We know that $k(x, y)$ is real-valued, hence the imaginary part on the right-side side is zero, hence: $e^{-\|x - y\|_2^2 / 2} = \mathbb{E}_{\omega} \left[\cos(\omega^T (x - y)) \right]$
- Recall the trig sum-product formula: $\cos(\alpha - \beta) = \cos \alpha \cos \beta + \sin \alpha \sin \beta$
- Define a real-valued 2-dimensional random feature: $f_{\omega}(x) = (\cos(\omega^T x), \sin(\omega^T x))$

Real Valued RFF: 2-Dim Random Features

- We have: $k(x, y) = \mathbb{E}_{\omega} \left[e^{i \cdot \omega^T (x - y)} \right] = \mathbb{E}_{\omega} \left[\cos(\omega^T (x - y)) + i \cdot \sin(\omega^T (x - y)) \right]$
- We know that $k(x, y)$ is real-valued, hence the imaginary part on the right-side side is zero, hence: $e^{-\|x-y\|_2^2/2} = \mathbb{E}_{\omega} \left[\cos(\omega^T (x - y)) \right]$
- Recall the trig sum-product formula: $\cos(\alpha - \beta) = \cos \alpha \cos \beta + \sin \alpha \sin \beta$
- Define a real-valued 2-dimensional random feature: $f_{\omega}(x) = (\cos(\omega^T x), \sin(\omega^T x))$
- Then: $f_{\omega}(x)^T f_{\omega}(y) = \cos(\omega^T x) \cos(\omega^T y) + \sin(\omega^T x) \sin(\omega^T y) = \cos(\omega^T (x - y))$

Real Valued RFF: 2-Dim Random Features

- We have: $k(x, y) = \mathbb{E}_{\omega} \left[e^{i \cdot \omega^T (x - y)} \right] = \mathbb{E}_{\omega} \left[\cos(\omega^T (x - y)) + i \cdot \sin(\omega^T (x - y)) \right]$
- We know that $k(x, y)$ is real-valued, hence the imaginary part on the right-side side is zero, hence: $e^{-\|x-y\|_2^2/2} = \mathbb{E}_{\omega} \left[\cos(\omega^T (x - y)) \right]$
- Recall the trig sum-product formula: $\cos(\alpha - \beta) = \cos \alpha \cos \beta + \sin \alpha \sin \beta$
- Define a real-valued 2-dimensional random feature: $f_{\omega}(x) = (\cos(\omega^T x), \sin(\omega^T x))$
- Then: $f_{\omega}(x)^T f_{\omega}(y) = \cos(\omega^T x) \cos(\omega^T y) + \sin(\omega^T x) \sin(\omega^T y) = \cos(\omega^T (x - y))$
- Thus: $\mathbb{E}_{\omega} [f_{\omega}(x)^T f_{\omega}(y)] = \mathbb{E}_{\omega} [\cos(\omega^T (x - y))] = e^{-\|x-y\|_2^2/2}$ as needed

Real Valued RFF: Scalar Random Features (1/2)

Real Valued RFF: Scalar Random Features (1/2)

- Recall another trig identity: $2 \cos(\alpha) \cos(\beta) = \cos(\alpha - \beta) + \cos(\alpha + \beta)$

Real Valued RFF: Scalar Random Features (1/2)

- Recall another trig identity: $2 \cos(\alpha) \cos(\beta) = \cos(\alpha - \beta) + \cos(\alpha + \beta)$
- Thus, for every $b \in \mathbb{R}$:

$$2 \cos(\omega^T x + b) \cos(\omega^T y + b) = \cos(\omega^T (x - y)) + \cos(\omega^T (x + y) + 2b)$$

Real Valued RFF: Scalar Random Features (1/2)

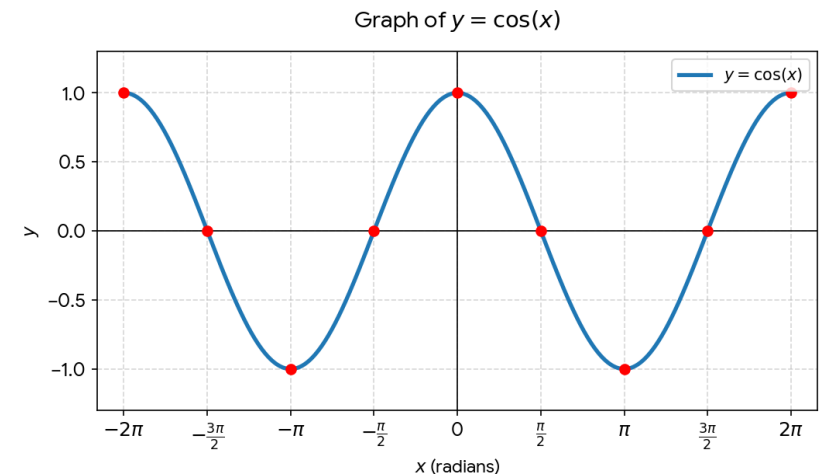
- Recall another trig identity: $2 \cos(\alpha) \cos(\beta) = \cos(\alpha - \beta) + \cos(\alpha + \beta)$
- Thus, for every $b \in \mathbb{R}$:

$$2 \cos(\omega^T x + b) \cos(\omega^T y + b) = \cos(\omega^T (x - y)) + \cos(\omega^T (x + y) + 2b)$$

- If we sample $b \sim [0, \pi)$ uniformly at random, then for every fixed z :

$$\mathbb{E}_b [\cos(\omega^T (x + y) + 2b)] = 0$$

Why?



Real Valued RFF: Scalar Random Features (1/2)

- Recall another trig identity: $2 \cos(\alpha) \cos(\beta) = \cos(\alpha - \beta) + \cos(\alpha + \beta)$
- Thus, for every $b \in \mathbb{R}$:

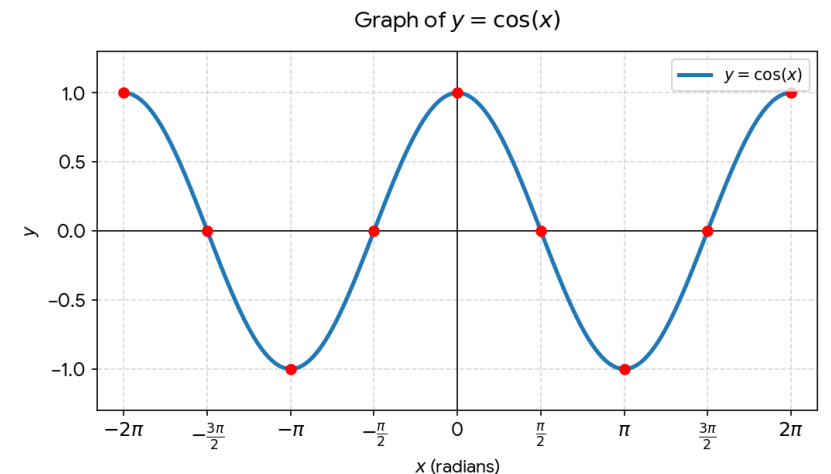
$$2 \cos(\omega^T x + b) \cos(\omega^T y + b) = \cos(\omega^T (x - y)) + \cos(\omega^T (x + y) + 2b)$$

- If we sample $b \sim [0, \pi)$ uniformly at random, then for every fixed z :

$$\mathbb{E}_b [\cos(\omega^T (x + y) + 2b)] = 0$$

Why?

- Hence, we define: $f_{\omega,b}(x) = \sqrt{2} \cos(\omega^T x + b)$



Real Valued RFF: Scalar Random Features (2/2)

- Putting things together:
- $\mathbb{E}_{\omega} [f_{\omega,b}(x)^T f_{\omega,b}(y)] = \mathbb{E}_{\omega,b} [2 \cos(\omega^T x + b) \cdot \cos(\omega^T y + b)]$
- $= \mathbb{E}_{\omega,b} [\cos(\omega^T (x - y)) + \cos(\omega^T (x + y) + 2b)]$
- $= \mathbb{E}_{\omega,b} [\cos(\omega^T (x - y))]$
- $= e^{-\|x-y\|_2^2/2}$

Real Valued RFF: Scalar Random Features (2/2)

- Putting things together:
- $\mathbb{E}_{\omega} [f_{\omega,b}(x)^T f_{\omega,b}(y)] = \mathbb{E}_{\omega,b} [2 \cos(\omega^T x + b) \cdot \cos(\omega^T y + b)]$
- $= \mathbb{E}_{\omega,b} [\cos(\omega^T (x - y)) + \cos(\omega^T (x + y) + 2b)]$
- $= \mathbb{E}_{\omega,b} [\cos(\omega^T (x - y))]$
- $= e^{-\|x-y\|_2^2/2}$
- Bottom line: To sample each random feature, we draw a gaussian $\omega \sim N(0,1)$ and a random shift $b \sim Uni[0,2\pi)$ and return $f_{\omega,b}(x) = \sqrt{2} \cos(\omega^T x + b)$

Real Valued RFF: Scalar Random Features (2/2)

- Putting things together:
- $\mathbb{E}_{\omega} [f_{\omega,b}(x)^T f_{\omega,b}(y)] = \mathbb{E}_{\omega,b} [2 \cos(\omega^T x + b) \cdot \cos(\omega^T y + b)]$
- $= \mathbb{E}_{\omega,b} [\cos(\omega^T (x - y)) + \cos(\omega^T (x + y) + 2b)]$
- $= \mathbb{E}_{\omega,b} [\cos(\omega^T (x - y))]$
- $= e^{-\|x-y\|_2^2/2}$
- Bottom line: To sample each random feature, we draw a gaussian $\omega \sim N(0,1)$ and a random shift $b \sim \text{Uni}[0, 2\pi)$ and return $f_{\omega,b}(x) = \sqrt{2} \cos(\omega^T x + b)$
- To get additive $k(x, y) \pm \varepsilon$ w.p. $1 - \delta$, how many features we draw and why?

RFF beyond the Gaussian Kernel

- Can we define RFF for other kernels? How general is the technique?

RFF beyond the Gaussian Kernel

- Can we define RFF for other kernels? How general is the technique?
- We need a few definitions:

RFF beyond the Gaussian Kernel

- Can we define RFF for other kernels? How general is the technique?
- We need a few definitions:
- **Definition 1:** a kernel k is **shift-invariant** if $k(x, y)$ depends only on $x - y$
 - The Gaussian kernel $e^{-\|x-y\|_2^2/2}$ clearly satisfies this
 - We will abuse notation and write $k(x, y) = k(x - y)$

RFF beyond the Gaussian Kernel

- Can we define RFF for other kernels? How general is the technique?
- We need a few definitions:
- **Definition 1:** a kernel k is **shift-invariant** if $k(x, y)$ depends only on $x - y$
 - The Gaussian kernel $e^{-\|x-y\|_2^2/2}$ clearly satisfies this
 - We will abuse notation and write $k(x, y) = k(x - y)$
- **Definition 2:** let $x_1, \dots, x_n \in \mathbb{R}^d$. The associated $n \times n$ **kernel matrix** is $K_{ij} = k(x_i, x_j)$

RFF beyond the Gaussian Kernel

- Can we define RFF for other kernels? How general is the technique?
- We need a few definitions:
- **Definition 1:** a kernel k is **shift-invariant** if $k(x, y)$ depends only on $x - y$
 - The Gaussian kernel $e^{-\|x-y\|_2^2/2}$ clearly satisfies this
 - We will abuse notation and write $k(x, y) = k(x - y)$
- **Definition 2:** let $x_1, \dots, x_n \in \mathbb{R}^d$. The associated $n \times n$ **kernel matrix** is $K_{ij} = k(x_i, x_j)$
- **Definition 3:** a kernel k is **positive definite** if
 1. It is symmetric: $k(x, y) = k(y, x)$ for every $x, y \in \mathbb{R}^d$
 2. The kernel matrix is positive semi-definite for every $x_1, \dots, x_n \in \mathbb{R}^d$

RFF beyond the Gaussian Kernel

- Can we define RFF for other kernels? How general is the technique?
- We need a few definitions:
- **Definition 1:** a kernel k is **shift-invariant** if $k(x, y)$ depends only on $x - y$
 - The Gaussian kernel $e^{-\|x-y\|_2^2/2}$ clearly satisfies this
 - We will abuse notation and write $k(x, y) = k(x - y)$
- **Definition 2:** let $x_1, \dots, x_n \in \mathbb{R}^d$. The associated $n \times n$ **kernel matrix** is $K_{ij} = k(x_i, x_j)$
- **Definition 3:** a kernel k is **positive definite** if
 1. It is symmetric: $k(x, y) = k(y, x)$ for every $x, y \in \mathbb{R}^d$
 2. The kernel matrix is positive semi-definite for every $x_1, \dots, x_n \in \mathbb{R}^d$
 - *Reminder: what is a PSD matrix?*

RFF beyond the Gaussian Kernel

- Can we define RFF for other kernels? How general is the technique?
- We need a few definitions:
- **Definition 1:** a kernel k is **shift-invariant** if $k(x, y)$ depends only on $x - y$
 - The Gaussian kernel $e^{-\|x-y\|_2^2/2}$ clearly satisfies this
 - We will abuse notation and write $k(x, y) = k(x - y)$
- **Definition 2:** let $x_1, \dots, x_n \in \mathbb{R}^d$. The associated $n \times n$ **kernel matrix** is $K_{ij} = k(x_i, x_j)$
- **Definition 3:** a kernel k is **positive definite** if
 1. It is symmetric: $k(x, y) = k(y, x)$ for every $x, y \in \mathbb{R}^d$
 2. The kernel matrix is positive semi-definite for every $x_1, \dots, x_n \in \mathbb{R}^d$
 - *Reminder: what is a PSD matrix?*
 - The Gaussian kernel is positive definite (**can you prove it now?**)

RFF beyond the Gaussian Kernel

- **Claim:** every positive definite shift invariant kernel has random Fourier features

RFF beyond the Gaussian Kernel

- **Claim:** every positive definite shift invariant kernel has random Fourier features
- This follows from [Bochner's Theorem](#): Every continuous shift invariant positive definite kernel k over $\mathbb{R}^d$ is the Fourier transform of some finite positive measure μ over $\mathbb{R}^d$:

$$k(x, y) = k(x - y) = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T (x - y)} \cdot d\mu(\omega)$$

RFF beyond the Gaussian Kernel

- **Claim:** every positive definite shift invariant kernel has random Fourier features
- This follows from [Bochner's Theorem](#): Every continuous shift invariant positive definite kernel k over $\mathbb{R}^d$ is the Fourier transform of some a finite positive measure μ over $\mathbb{R}^d$:

$$k(x, y) = k(x - y) = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T (x - y)} \cdot d\mu(\omega)$$

- We can normalize k so that μ is a probability measure, and then:

$$k(x, y) = \mathbb{E}_{\omega} \left[e^{i \cdot \omega^T (x - y)} \right]$$

RFF beyond the Gaussian Kernel

- **Claim:** every positive definite shift invariant kernel has random Fourier features
- This follows from **Bochner's Theorem**: Every continuous shift invariant positive definite kernel k over $\mathbb{R}^d$ is the Fourier transform of some finite positive measure μ over $\mathbb{R}^d$:

$$k(x, y) = k(x - y) = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T (x - y)} \cdot d\mu(\omega)$$

- We can normalize k so that μ is a probability measure, and then:

$$k(x, y) = \mathbb{E}_{\omega} \left[e^{i \cdot \omega^T (x - y)} \right]$$

- Same construction(s): $f_{\omega}(x) = (\cos(\omega^T x), \sin(\omega^T x))$ and $f_{\omega, b}(x) = \sqrt{2} \cos(\omega^T x + b)$
 - The only thing that changes is that ω is sampled from μ

RFF beyond the Gaussian Kernel

- **Claim:** every positive definite shift invariant kernel has random Fourier features
- This follows from **Bochner's Theorem**: Every continuous shift invariant positive definite kernel k over $\mathbb{R}^d$ is the Fourier transform of some finite positive measure μ over $\mathbb{R}^d$:

$$k(x, y) = k(x - y) = \int_{\mathbb{R}^d} e^{-i \cdot \omega^T (x - y)} \cdot d\mu(\omega)$$

- We can normalize k so that μ is a probability measure, and then:

$$k(x, y) = \mathbb{E}_{\omega} \left[e^{i \cdot \omega^T (x - y)} \right]$$

- Same construction(s): $f_{\omega}(x) = (\cos(\omega^T x), \sin(\omega^T x))$ and $f_{\omega, b}(x) = \sqrt{2} \cos(\omega^T x + b)$
 - The only thing that changes is that ω is sampled from μ
 - Note: the fact that μ exists doesn't mean we can find it or efficiently sample from it

LSH for Kernels

A.k.a, “Random Binning Features”

Reminder: LSH for Sketching Similarities

- Recall our definition from Lecture 2 of [LSH for sketching similarities](#):
 - Let X be a space of elements with a similarity measure $k: X \times X \rightarrow \mathbb{R}$
 - An LSH scheme for k over X is a distribution $\mathcal{H}$ over hash functions h that map each element of X into one of B bins such that for all $x, y \in X$:

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(y)] = k(x, y)$$

Reminder: LSH for Sketching Similarities

- Recall our definition from Lecture 2 of LSH for sketching similarities:
 - Let X be a space of elements with a similarity measure $k: X \times X \rightarrow \mathbb{R}$
 - An LSH scheme for k over X is a distribution $\mathcal{H}$ over hash functions h that map each element of X into one of B bins such that for all $x, y \in X$:

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(y)] = k(x, y)$$

- We saw two examples – what similarities did we sketch?

Reminder: LSH for Sketching Similarities

- Recall our definition from Lecture 2 of LSH for sketching similarities:
 - Let X be a space of elements with a similarity measure $k: X \times X \rightarrow \mathbb{R}$
 - An LSH scheme for k over X is a distribution $\mathcal{H}$ over hash functions h that map each element of X into one of B bins such that for all $x, y \in X$:

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(y)] = k(x, y)$$

- We saw two examples – what similarities did we sketch?

MinHash for the Jaccard similarity over sets:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Reminder: LSH for Sketching Similarities

- Recall our definition from Lecture 2 of LSH for sketching similarities:
 - Let X be a space of elements with a similarity measure $k: X \times X \rightarrow \mathbb{R}$
 - An LSH scheme for k over X is a distribution $\mathcal{H}$ over hash functions h that map each element of X into one of B bins such that for all $x, y \in X$:

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(y)] = k(x, y)$$

- We saw two examples – what similarities did we sketch?

MinHash for the Jaccard similarity over sets:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

SimHash for the “angular kernel” over a sphere:

$$k(x, y) = \frac{\text{angle between } x \text{ and } y}{\pi}$$

Reminder: LSH for Sketching Similarities

- Recall our definition from Lecture 2 of LSH for sketching similarities:
 - Let X be a space of elements with a similarity measure $k: X \times X \rightarrow \mathbb{R}$
 - An LSH scheme for k over X is a distribution $\mathcal{H}$ over hash functions h that map each element of X into one of B bins such that for all $x, y \in X$:

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(y)] = k(x, y)$$

- We saw two examples – what similarities did we sketch?

MinHash for the Jaccard similarity over sets:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

What were LSH schemes?

SimHash for the “angular kernel” over a sphere:

$$k(x, y) = \frac{\text{angle between } x \text{ and } y}{\pi}$$

LSH as Random Features

- Recall our definition of **random features**: a distribution $\mathcal{F}$ over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all x, y :

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

LSH as Random Features

- Recall our definition of **random features**: a distribution $\mathcal{F}$ over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all x, y :

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Recall our definition for **LSH** for sketching similarities: distribution $\mathcal{H}$ over hash functions into B bins such that for all x, y :

$$\Pr_{h \sim \mathcal{H}}[h(x) = h(y)] = k(x, y)$$

LSH as Random Features

- Recall our definition of **random features**: a distribution $\mathcal{F}$ over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all x, y :

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Recall our definition for **LSH** for sketching similarities: distribution $\mathcal{H}$ over hash functions into B bins such that for all x, y :

$$\Pr_{h \sim \mathcal{H}}[h(x) = h(y)] = k(x, y)$$

- Can we turn an LSH scheme for k into random features?

LSH as Random Features

- Recall our definition of **random features**: a distribution $\mathcal{F}$ over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all x, y :

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Recall our definition for **LSH** for sketching similarities: distribution $\mathcal{H}$ over hash functions into B bins such that for all x, y :

$$\Pr_{h \sim \mathcal{H}}[h(x) = h(y)] = k(x, y)$$

- Can we turn an LSH scheme for k into random features?
- For every h define the random feature $f_h: \mathbb{R}^d \rightarrow \mathbb{R}^B$ as $f_h(x)_j :=$
 - i.e., $f_h(x)$ is the one-hot vector of the bin x is hashed to

???

LSH as Random Features

- Recall our definition of **random features**: a distribution $\mathcal{F}$ over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all x, y :

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Recall our definition for **LSH** for sketching similarities: distribution $\mathcal{H}$ over hash functions into B bins such that for all x, y :

$$\Pr_{h \sim \mathcal{H}}[h(x) = h(y)] = k(x, y)$$

- Can we turn an LSH scheme for k into random features?
- For every h define the random feature $f_h: \mathbb{R}^d \rightarrow \mathbb{R}^B$ as $f_h(x)_j := \mathbf{1}\{h(x) = j\}$
 - i.e., $f_h(x)$ is the one-hot vector of the bin x is hashed to

LSH as Random Features

- Recall our definition of **random features**: a distribution $\mathcal{F}$ over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all x, y :

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Recall our definition for **LSH** for sketching similarities: distribution $\mathcal{H}$ over hash functions into B bins such that for all x, y :

$$\Pr_{h \sim \mathcal{H}}[h(x) = h(y)] = k(x, y)$$

- Can we turn an LSH scheme for k into random features?
- For every h define the random feature $f_h: \mathbb{R}^d \rightarrow \mathbb{R}^B$ as $f_h(x)_j := \mathbf{1}\{h(x) = j\}$
 - i.e., $f_h(x)$ is the one-hot vector of the bin x is hashed to
- Then: $\mathbb{E}_{h \sim \mathcal{H}}[f_h(x)^T f_h(y)] = \mathbb{E}_{h \sim \mathcal{H}}[\mathbf{1}\{h(x) = h(y)\}] = k(x, y)$

LSH as Random Features

- Recall our definition of **random features**: a distribution $\mathcal{F}$ over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all x, y :

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Recall our definition for **LSH** for sketching similarities: distribution $\mathcal{H}$ over hash functions into B bins such that for all x, y :

$$\Pr_{h \sim \mathcal{H}}[h(x) = h(y)] = k(x, y)$$

- Can we turn an LSH scheme for k into random features?
- For every h define the random feature $f_h: \mathbb{R}^d \rightarrow \mathbb{R}^B$ as $f_h(x)_j := \mathbf{1}\{h(x) = j\}$
 - i.e., $f_h(x)$ is the one-hot vector of the bin x is hashed to
- Then: $\mathbb{E}_{h \sim \mathcal{H}}[f_h(x)^T f_h(y)] = \mathbb{E}_{h \sim \mathcal{H}}[\mathbf{1}\{h(x) = h(y)\}] = k(x, y)$

See a big problem here?

LSH as Random Features

- Recall our definition of **random features**: a distribution $\mathcal{F}$ over functions $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that for all x, y :

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- Recall our definition for **LSH** for sketching similarities: distribution $\mathcal{H}$ over hash functions into B bins such that for all x, y :

$$\Pr_{h \sim \mathcal{H}}[h(x) = h(y)] = k(x, y)$$

- Can we turn an LSH scheme for k into random features?
- For every h define the random feature $f_h: \mathbb{R}^d \rightarrow \mathbb{R}^B$ as $f_h(x)_j := \mathbf{1}\{h(x) = j\}$
 - i.e., $f_h(x)$ is the one-hot vector of the bin x is hashed to
- Then: $\mathbb{E}_{h \sim \mathcal{H}}[f_h(x)^T f_h(y)] = \mathbb{E}_{h \sim \mathcal{H}}[\mathbf{1}\{h(x) = h(y)\}] = k(x, y)$

See a big problem here?

See a solution?

Rehashing

- Let B be the number of hash bins in the range of the LSH function in $\mathcal{H}$ (it can be ∞)
- Let M our desired number of bins

Rehashing

- Let B be the number of hash bins in the range of the LSH function in $\mathcal{H}$ (it can be ∞)
- Let M our desired number of bins
- Let $\mathcal{U}$ a family of pairwise independent hash function $u: [B] \rightarrow [M]$
 - For all $j_1, j_2 \in [B]$ and $\ell_1, \ell_2 \in [M]$: $\Pr_{u \sim \mathcal{U}}[u(j_1) = \ell_1 \text{ and } u(j_2) = \ell_2] = 1/M^2$

Rehashing

- Let B be the number of hash bins in the range of the LSH function in $\mathcal{H}$ (it can be ∞)
- Let M our desired number of bins
- Let $\mathcal{U}$ a family of pairwise independent hash function $u: [B] \rightarrow [M]$
 - For all $j_1, j_2 \in [B]$ and $\ell_1, \ell_2 \in [M]$: $\Pr_{u \sim \mathcal{U}}[u(j_1) = \ell_1 \text{ and } u(j_2) = \ell_2] = 1/M^2$
- For every $h \in \mathcal{H}$ and $u \in \mathcal{U}$, we define the random feature $f_{h,u}: \mathbb{R}^d \rightarrow \mathbb{R}^M$ as

$$f_{h,u}(x)_\ell := \mathbf{1}\{u(h(x)) = \ell\}$$

Rehashing

- Let B be the number of hash bins in the range of the LSH function in $\mathcal{H}$ (it can be ∞)
- Let M our desired number of bins
- Let $\mathcal{U}$ a family of pairwise independent hash function $u: [B] \rightarrow [M]$
 - For all $j_1, j_2 \in [B]$ and $\ell_1, \ell_2 \in [M]$: $\Pr_{u \sim \mathcal{U}} [u(j_1) = \ell_1 \text{ and } u(j_2) = \ell_2] = 1/M^2$
- For every $h \in \mathcal{H}$ and $u \in \mathcal{U}$, we define the random feature $f_{h,u}: \mathbb{R}^d \rightarrow \mathbb{R}^M$ as

$$f_{h,u}(x)_\ell := \mathbf{1}\{u(h(x)) = \ell\}$$

- What could cause a collision? Either h , or u

Rehashing

- Let B be the number of hash bins in the range of the LSH function in $\mathcal{H}$ (it can be ∞)
- Let M our desired number of bins
- Let $\mathcal{U}$ a family of pairwise independent hash function $u: [B] \rightarrow [M]$
 - For all $j_1, j_2 \in [B]$ and $\ell_1, \ell_2 \in [M]$: $\Pr_{u \sim \mathcal{U}}[u(j_1) = \ell_1 \text{ and } u(j_2) = \ell_2] = 1/M^2$
- For every $h \in \mathcal{H}$ and $u \in \mathcal{U}$, we define the random feature $f_{h,u}: \mathbb{R}^d \rightarrow \mathbb{R}^M$ as

$$f_{h,u}(x)_\ell := \mathbf{1}\{u(h(x)) = \ell\}$$

- What could cause a collision? Either h , or u
- By a union bound: $\Pr_{h,u}[u(h(x)) = u(h(y))] \leq k(x, y) + \frac{1}{M}$

Rehashing

- Let B be the number of hash bins in the range of the LSH function in $\mathcal{H}$ (it can be ∞)
- Let M our desired number of bins
- Let $\mathcal{U}$ a family of pairwise independent hash function $u: [B] \rightarrow [M]$
 - For all $j_1, j_2 \in [B]$ and $\ell_1, \ell_2 \in [M]$: $\Pr_{u \sim \mathcal{U}}[u(j_1) = \ell_1 \text{ and } u(j_2) = \ell_2] = 1/M^2$
- For every $h \in \mathcal{H}$ and $u \in \mathcal{U}$, we define the random feature $f_{h,u}: \mathbb{R}^d \rightarrow \mathbb{R}^M$ as

$$f_{h,u}(x)_\ell := \mathbf{1}\{u(h(x)) = \ell\}$$

- What could cause a collision? Either h , or u
- By a union bound: $\Pr_{h,u}[u(h(x)) = u(h(y))] \leq k(x, y) + \frac{1}{M}$
- Hence: $\mathbb{E}_{h \sim \mathcal{H}, u \sim \mathcal{U}}[f_h(x)^T f_h(y)] = k(x, y) \pm \frac{1}{M}$

Which Kernels Have LSH Schemes?

Which Kernels Have LSH Schemes?

- **Theorem** [Charikar'02]: Let k be a kernel with an LSH scheme $\mathcal{H}$. Then, $1 - k$ satisfies the triangle inequality: $\forall x, y, z, 1 - k(x, y) \leq 1 - k(x, z) + 1 - k(y, z)$.

Which Kernels Have LSH Schemes?

- **Theorem** [Charikar'02]: Let k be a kernel with an LSH scheme $\mathcal{H}$. Then, $1 - k$ satisfies the triangle inequality: $\forall x, y, z, 1 - k(x, y) \leq 1 - k(x, z) + 1 - k(y, z)$.
- Proof:
 - Fix $h \in \mathcal{H}$.

Which Kernels Have LSH Schemes?

- **Theorem** [Charikar'02]: Let k be a kernel with an LSH scheme $\mathcal{H}$. Then, $1 - k$ satisfies the triangle inequality: $\forall x, y, z, 1 - k(x, y) \leq 1 - k(x, z) + 1 - k(y, z)$.
- Proof:
 - Fix $h \in \mathcal{H}$.
 - Claim: $\mathbf{1}\{h(x) \neq h(y)\}$ satisfies the triangle inequality, i.e.,
$$\forall x, y, z, \quad \mathbf{1}\{h(x) \neq h(y)\} \leq \mathbf{1}\{h(x) \neq h(z)\} + \mathbf{1}\{h(y) \neq h(z)\}$$

Which Kernels Have LSH Schemes?

- **Theorem** [Charikar'02]: Let k be a kernel with an LSH scheme $\mathcal{H}$. Then, $1 - k$ satisfies the triangle inequality: $\forall x, y, z, 1 - k(x, y) \leq 1 - k(x, z) + 1 - k(y, z)$.

- Proof:

- Fix $h \in \mathcal{H}$.

- Claim: $\mathbf{1}\{h(x) \neq h(y)\}$ satisfies the triangle inequality, i.e.,

$$\forall x, y, z, \quad \mathbf{1}\{h(x) \neq h(y)\} \leq \mathbf{1}\{h(x) \neq h(z)\} + \mathbf{1}\{h(y) \neq h(z)\}$$

Proof?

Which Kernels Have LSH Schemes?

- **Theorem** [Charikar'02]: Let k be a kernel with an LSH scheme $\mathcal{H}$. Then, $1 - k$ satisfies the triangle inequality: $\forall x, y, z, 1 - k(x, y) \leq 1 - k(x, z) + 1 - k(y, z)$.
- Proof:
 - Fix $h \in \mathcal{H}$.
 - Claim: $\mathbf{1}\{h(x) \neq h(y)\}$ satisfies the triangle inequality, i.e.,
$$\forall x, y, z, \quad \mathbf{1}\{h(x) \neq h(y)\} \leq \mathbf{1}\{h(x) \neq h(z)\} + \mathbf{1}\{h(y) \neq h(z)\}$$
 - Proof: The only case to check is where the right-hand side is zero. In that case, $h(x) = h(z)$ and $h(y) = h(z)$, thus $h(x) = h(y)$, thus $\mathbf{1}\{h(x) \neq h(y)\} = 0$.

Which Kernels Have LSH Schemes?

- **Theorem** [Charikar'02]: Let k be a kernel with an LSH scheme $\mathcal{H}$. Then, $1 - k$ satisfies the triangle inequality: $\forall x, y, z, 1 - k(x, y) \leq 1 - k(x, z) + 1 - k(y, z)$.

- Proof:

- Fix $h \in \mathcal{H}$.

- Claim: $\mathbf{1}\{h(x) \neq h(y)\}$ satisfies the triangle inequality, i.e.,

$$\forall x, y, z, \quad \mathbf{1}\{h(x) \neq h(y)\} \leq \mathbf{1}\{h(x) \neq h(z)\} + \mathbf{1}\{h(y) \neq h(z)\}$$

- Proof: The only case to check is where the right-hand side is zero. In that case, $h(x) = h(z)$ and $h(y) = h(z)$, thus $h(x) = h(y)$, thus $\mathbf{1}\{h(x) \neq h(y)\} = 0$.

What
now?

Which Kernels Have LSH Schemes?

- **Theorem** [Charikar'02]: Let k be a kernel with an LSH scheme $\mathcal{H}$. Then, $1 - k$ satisfies the triangle inequality: $\forall x, y, z, 1 - k(x, y) \leq 1 - k(x, z) + 1 - k(y, z)$.

- Proof:

- Fix $h \in \mathcal{H}$.

- Claim: $\mathbf{1}\{h(x) \neq h(y)\}$ satisfies the triangle inequality, i.e.,

$$\forall x, y, z, \quad \mathbf{1}\{h(x) \neq h(y)\} \leq \mathbf{1}\{h(x) \neq h(z)\} + \mathbf{1}\{h(y) \neq h(z)\}$$

- Proof: The only case to check is where the right-hand side is zero. In that case, $h(x) = h(z)$ and $h(y) = h(z)$, thus $h(x) = h(y)$, thus $\mathbf{1}\{h(x) \neq h(y)\} = 0$.
- Since $\mathbb{E}_{h \sim \mathcal{H}}[\mathbf{1}\{h(x) \neq h(y)\}] = 1 - k(x, y)$, the theorem follows by taking expectation over the claim.



Some Consequences

- **Theorem** [Charikar'02]: Let k be a kernel with an LSH scheme $\mathcal{H}$. Then, $1 - k$ satisfies the triangle inequality: $\forall x, y, z, 1 - k(x, y) \leq 1 - k(x, z) + 1 - k(y, z)$.

Some Consequences

- **Theorem** [Charikar'02]: Let k be a kernel with an LSH scheme $\mathcal{H}$. Then, $1 - k$ satisfies the triangle inequality: $\forall x, y, z, 1 - k(x, y) \leq 1 - k(x, z) + 1 - k(y, z)$.
- **Corollary 1**: Let $J(A, B)$ denote the Jaccard similarity between sets. Then $1 - J(A, B)$ is a metric.

Some Consequences

- **Theorem** [Charikar'02]: Let k be a kernel with an LSH scheme $\mathcal{H}$. Then, $1 - k$ satisfies the triangle inequality: $\forall x, y, z, 1 - k(x, y) \leq 1 - k(x, z) + 1 - k(y, z)$.
- **Corollary 1**: Let $J(A, B)$ denote the Jaccard similarity between sets. Then $1 - J(A, B)$ is a metric.
- **Corollary 2**: No LSH scheme exists for the Gaussian kernel $k(x, y) = e^{-\|x-y\|_2^2/\sigma^2}$.
 - (Requires proof; we won't prove for now)

LSH for the Laplacian Kernel

- **Theorem:** The Laplacian kernel $k(x, y) = e^{-\|x-y\|_1/\sigma}$ has an LSH scheme.

LSH for the Laplacian Kernel

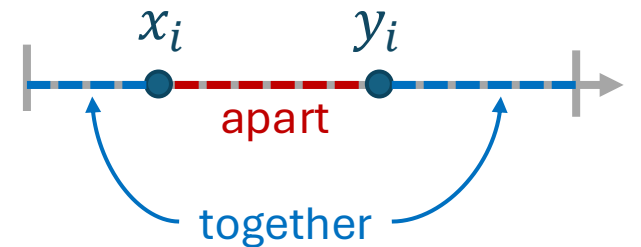
- **Theorem:** The Laplacian kernel $k(x, y) = e^{-\|x-y\|_1/\sigma}$ has an LSH scheme.
 - We will see two different LSH constructions
 - For the first one, we assume that all points have coordinates in $[0,1]$

LSH for the Laplacian Kernel

- **Theorem:** The Laplacian kernel $k(x, y) = e^{-\|x-y\|_1/\sigma}$ has an LSH scheme.
 - We will see two different LSH constructions
 - For the first one, we assume that all points have coordinates in $[0,1]$
- **Step 1:**
 - Fix an axis $i \in [d]$. Choose a uniformly random threshold $\tau \sim [0,1]$.
 - Define $h(x) = 1$ if $x_i > \tau$ and $h(x) = 0$ otherwise.

LSH for the Laplacian Kernel

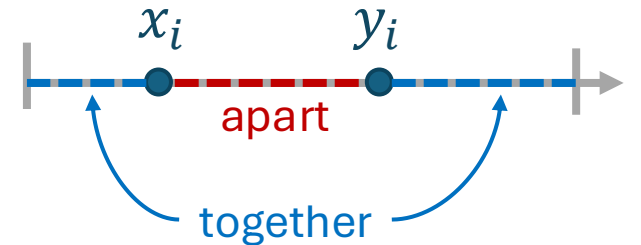
- **Theorem:** The Laplacian kernel $k(x, y) = e^{-\|x-y\|_1/\sigma}$ has an LSH scheme.
 - We will see two different LSH constructions
 - For the first one, we assume that all points have coordinates in $[0,1]$
- **Step 1:**
 - Fix an axis $i \in [d]$. Choose a uniformly random threshold $\tau \sim [0,1]$.
 - Define $h(x) = 1$ if $x_i > \tau$ and $h(x) = 0$ otherwise.
 - Collision probability? (Yes, this again)



LSH for the Laplacian Kernel

- **Theorem:** The Laplacian kernel $k(x, y) = e^{-\|x-y\|_1/\sigma}$ has an LSH scheme.
 - We will see two different LSH constructions
 - For the first one, we assume that all points have coordinates in $[0,1]$
- **Step 1:**
 - Fix an axis $i \in [d]$. Choose a uniformly random threshold $\tau \sim [0,1]$.
 - Define $h(x) = 1$ if $x_i > \tau$ and $h(x) = 0$ otherwise.
 - **Collision probability? (Yes, this again)**

$$\Pr_{\tau}[h(x) = h(y)] = 1 - |x_i - y_i|$$



LSH for the Laplacian Kernel

- Step 2: Now, choose the axis $i \sim [d]$ uniformly at random.
 - Collision probability?

LSH for the Laplacian Kernel

- **Step 2:** Now, choose the axis $i \sim [d]$ uniformly at random.
 - Collision probability?

$$\Pr_{i,\tau}[h(x) = h(y)] = \sum_{i=1}^d \Pr[i] \cdot \Pr_{\tau}[h(x) = h(y) \mid i]$$

LSH for the Laplacian Kernel

- Step 2: Now, choose the axis $i \sim [d]$ uniformly at random.
 - Collision probability?

$$\begin{aligned}\Pr_{i,\tau}[h(x) = h(y)] &= \sum_{i=1}^d \Pr[i] \cdot \Pr_{\tau}[h(x) = h(y) \mid i] \\ &= \sum_{i=1}^d \frac{1}{d} \cdot (1 - |x_i - y_i|) = \boxed{???}\end{aligned}$$

LSH for the Laplacian Kernel

- Step 2: Now, choose the axis $i \sim [d]$ uniformly at random.
 - Collision probability?

$$\begin{aligned}\Pr_{i,\tau}[h(x) = h(y)] &= \sum_{i=1}^d \Pr[i] \cdot \Pr_{\tau}[h(x) = h(y) \mid i] \\ &= \sum_{i=1}^d \frac{1}{d} \cdot (1 - |x_i - y_i|) = 1 - \|x - y\|_1\end{aligned}$$

LSH for the Laplacian Kernel

- **Step 2:** Now, choose the axis $i \sim [d]$ uniformly at random.
 - Collision probability?

$$\begin{aligned}\Pr_{i,\tau}[h(x) = h(y)] &= \sum_{i=1}^d \Pr[i] \cdot \Pr_{\tau}[h(x) = h(y) \mid i] \\ &= \sum_{i=1}^d \frac{1}{d} \cdot (1 - |x_i - y_i|) = 1 - \|x - y\|_1\end{aligned}$$

- **Step 3:** Now, concatenate r i.i.d. repetitions of the above hashing scheme.
 - Collision probability?

LSH for the Laplacian Kernel

- **Step 2:** Now, choose the axis $i \sim [d]$ uniformly at random.
 - Collision probability?

$$\begin{aligned}\Pr_{i,\tau}[h(x) = h(y)] &= \sum_{i=1}^d \Pr[i] \cdot \Pr_{\tau}[h(x) = h(y) \mid i] \\ &= \sum_{i=1}^d \frac{1}{d} \cdot (1 - |x_i - y_i|) = 1 - \|x - y\|_1\end{aligned}$$

- **Step 3:** Now, concatenate r i.i.d. repetitions of the above hashing scheme.
 - Collision probability?

$$\Pr_{i,\tau}[h(x) = h(y)] = (1 - \|x - y\|_1)^r$$

LSH for the Laplacian Kernel

- Step 4: Finally, sample the number of i.i.d. repetitions as $r \sim \text{Poisson}(d/\sigma)$.
 - Collision probability?

LSH for the Laplacian Kernel

- Step 4: Finally, sample the number of i.i.d. repetitions as $r \sim \text{Poisson}(d/\sigma)$.
 - Collision probability?

$$\Pr[h(x) = h(y)] = \sum_{r=0}^{\infty} \Pr[r] \cdot \Pr[h(x) = h(y) \mid r]$$

LSH for the Laplacian Kernel

- **Step 4:** Finally, sample the number of i.i.d. repetitions as $r \sim \text{Poisson}(d/\sigma)$.
 - Collision probability?

$$\begin{aligned}\Pr[h(x) = h(y)] &= \sum_{r=0}^{\infty} \Pr[r] \cdot \Pr[h(x) = h(y) \mid r] \\ &= \sum_{r=0}^{\infty} \frac{e^{-d/\sigma} (d/\sigma)^r}{r!} \left(1 - \frac{1}{d} \|x - y\|_1\right)^r\end{aligned}$$

LSH for the Laplacian Kernel

- **Step 4:** Finally, sample the number of i.i.d. repetitions as $r \sim \text{Poisson}(d/\sigma)$.
 - Collision probability?

$$\begin{aligned}\Pr[h(x) = h(y)] &= \sum_{r=0}^{\infty} \Pr[r] \cdot \Pr[h(x) = h(y) \mid r] \\ &= \sum_{r=0}^{\infty} \frac{e^{-d/\sigma} (d/\sigma)^r}{r!} \left(1 - \frac{1}{d} \|x - y\|_1\right)^r \\ &= e^{-\frac{d}{\sigma}} \cdot e^{\frac{d}{\sigma} \left(1 - \frac{1}{d} \|x - y\|_1\right)} = e^{-\frac{\|x - y\|_1}{\sigma}}\end{aligned}$$



LSH for the Laplacian Kernel: Conclusion

- Let's recap the steps of sampling an LSH scheme:
 1. Sample $r \sim \text{Poisson}(d/\sigma)$.
 2. Sample r i.i.d. axes $i_1, \dots, i_r$ uniformly at random from $\{1, \dots, d\}$.
 3. Sample r i.i.d. thresholds $\tau_1, \dots, \tau_r$ uniformly at random from $[0,1]$.
 4. The resulting hash function $h: \mathbb{R}^d \rightarrow \{0,1\}^r$ is defined as the concatenation of thresholding bits:

$$h(x) = \mathbf{1}\{x_{i_1} > t_1\} \odot \mathbf{1}\{x_{i_2} > t_2\} \odot \dots \odot \mathbf{1}\{x_{i_r} > t_r\}$$

Conclusion

- Kernels in high-dimensional spaces can be efficiently and approximately handled through **random features**
- We have seen two approaches to random features:
 - **Random Fourier Features**
 - **LSH**
- Both have many applications in machine learning
 - We will see a few later in class

Questions?